Received 1 September 2016; revised 30 May 2017; accepted 26 July 2017.
Date of publication 9 August 2017; date of current version 9 June 2020.

Digital Object Identifier 10.1109/TETC.2017.2737045

LEXACT: Low Energy N-Modular Redundancy Using Approximate Computing for Real-Time Multicore Processors

FARSHAD BAHARVAND AND SEYED GHASSEM MIREMADI

The authors are with the Department of Computer Engineering, Sharif University of Technology, Tehran 11365, Iran
CORRESPONDING AUTHOR: S. G. MIREMADI (miremadi@sharif.edu)

ABSTRACT Multicore processors are becoming popular in safety-critical applications. A series of these applications comprises of kernels where inexact computations may produce results within the boundary of sufficient quality though, for which the reliability should stay at the maximum possible level. Intrinsic core-level redundancy in multicore processors can be leveraged to achieve the desired reliability level in form of N-modular redundancy (NMR). While NMR provides a proactive means of reliability for critical systems, it has two main drawbacks: Increase in the area and energy consumption that are both limiting factors in the embedded systems. This paper presents a software-based method to construct NMR in a multicore processor through k cores using approximate computing concept, where $k < N$. The method uses a combination of exact and approximate versions of a task to run on different cores in order to increase the reliability. To identify the suitable approximation factor, a tool called LEXACT has been developed to process the target applications. The method reduces the energy consumption and area overheads of the NMR as compared to its traditional implementation. Experimental results show at least 35 percent reduction in the energy consumption, and some 40 percent area reduction while achieving the desired reliability level and quality of result.

INDEX TERMS Multicore processors, reliability, approximate computing, real-time embedded systems, N-modular redundancy

I. INTRODUCTION

The increasing number of independent software components in the emerging safety-critical embedded applications has made multicore processors one of the attractive choices for parallel execution of these applications. This parallelism provides further computational performance at lower power and cost [1]. An example in automotive domain is AUTOSAR [2]; a commonly-accepted framework for design of automotive applications. This framework is promoting multicore processors as the target platform for development of the new generation of electronic control units (ECU) [3].

In addition to the performance benefit mentioned above, multicore processors are suitable for implementing spatial redundancies for fault tolerance purposes due to their intrinsic redundancy. N-modular redundancy, as a fault tolerance method, can be realized in a multicore processor using N copies of the same task running concurrently on different cores. While improving the reliability, this type of redundancy however, consumes the available processing budget of the cores. Therefore, it becomes necessary to overcalculate this budget when designing for reliability which results in employing more cores. Additionally, any increase in the number of tasks due to the NMR results in more energy consumption which is an important constraint in the embedded systems. To this end, proposing methods that reduce the overhead of NMR in terms of energy consumption and additional cores required, have been the focus of the recent research in the field of multicore processors; as discussed in [4], [5], and [6].

Energy consumption of a task is proportional to its power consumption and execution time. A method discussed in [7] uses clock frequency management in favor of reducing execution time of the task. It aims at implementing NMR without increasing the number of cores. However, it leads to an increase in the power consumption of the task that may lead to more energy consumption again. Another method to reduce the execution time of a task without elevating the frequency is to use inexact computation [8]. This method,

2168-6750 © 2017 IEEE. Translations and content mining are permitted for academic research only.
Personal use is also permitted, but republication/redistribution requires IEEE permission.
See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.
Authorized licensed use limited to: UNIV OF HAWAII LIBRARY. Downloaded on November 08,2025 at 00:26:35 UTC from IEEE Xplore. Restrictions apply.

of course, is applicable to a particular range of applications. This concept is known as *approximate computing* that is the basis of the present research work.

Quality versus Reliability. Inexact or approximate computing may affect the quality of result of a task. However, for a range of critical applications, having a specific level of reliability is more important than generating a precise result. As an example, cameras in an autonomous car, continuously receive images from the surrounding environment, which need to be analyzed for extraction of important features for safety purposes; e.g., preventing from lane departure. A human driver does not need all the details of the images so neither does the autonomous system. However, the critical information must be reliably extracted from the images should a fault occur in the system. Therefore, the reliability in this case has a higher importance than exact computation of the unnecessary details. Moreover, one of the important characteristics of the safety-critical systems is their timeliness. Even if the system produces the correct output but it fails to deliver this output on time, then the safety-critical system has failed and the outcome could be catastrophic. A fault masking mechanism like NMR, though imposing more redundancy on the system, has negligible time overhead compared to a check and repair mechanism. Therefore, the NMR system might be a good choice when an intrinsic parallel redundancy (cores in a multicore system) exists.

This paper presents a low-overhead (in terms of area and energy) NMR-style core-level redundancy for improving reliability of the real-time multicore embedded processors based on the concept of approximate computing. By employing this method, in addition to increasing the reliability of a system, only k cores are used as compared to N cores deployed by a traditional NMR, where $k < N$. Redundant tasks are executed approximately to reduce their execution time and energy consumption. By reducing the execution time of the redundant tasks, the likelihood of executing them on a less number of cores before the deadline, increases as well. Therefore, by using this method, the area and energy overheads incurred by NMR method can be reduced significantly.

For the purpose of applying approximation methods on a set of selected applications, a tool called LEXACT has been developed that tries to find the suitable approximation factor that satisfies constraints such as reliability, quality of result, energy, and timeliness for an embedded system.

It is worth mentioning that, the proposed method covers applications that benefit from inexact computation. The method may be used on a general-purpose multicore hardware without requiring additional modification to the hardware. The software developer, who is aware of the system's applications, needs to profile the program before boarding the software on a multicore platform. The developer provides inexact version of the critical sections of the program alongside the exact version of the whole software.

The main contributions of this work are:

- A low-overhead (in terms of energy and area) NMR-style core-level method for improving reliability of multicore real-time systems is presented that uses a combination of approximate and exact computing.
- A tool called LEXACT is developed to mark the approximable sections of the target application by monitoring the quality of results. This can help application developer automate the process of production of the inexact version.
- Effect of fault injection on the input data for the inexact versions of the selected applications is investigated to optimize the approximation factor, considering quality of result and the produced reliability level.
- The proposed method is applied to the system at the highest level of abstraction compared to microarchitecture-based or circuit-level methods, and hence providing a general and more effective overhead reduction.

For the purpose of evaluation, part of *AxBench* benchmark suite [9] is used. This suite collects a wide-range of representative applications that can benefit from approximate computing. Additionally, some applications from *Mibench* suite [10], which is targeted for embedded applications have been also selected.

This paper is organized as follows: Section II provides a survey of the previous research work. In Section III, an introduction to the approximate computing concept and the related methods is presented. Section IV briefly explains the system models. Section V discusses the proposed method and the developed tool. It also provides some equations and constraints as the criteria for evaluation of the method. Section VI is devoted to the evaluation of the method and related discussions. Finally, Section VII concludes the paper.

II. RELATED WORK

This section briefly discusses previous research work that have employed approximate computing concept in conjunction with the reliability improvement methods in a system. To the best of our knowledge, this work is the first to use the approximate computing concept to optimize the N-modular redundancy in multicore processors in terms of core utilization and energy consumption while meeting the deadline constraints.

The work presented in [11] introduces an approximate voting scheme for dual modular redundancy that reduces the exact comparison overhead. Authors state that their technique does not resort to TMR, and hence saving the area and energy. However, when the probability of failure in a core is low, the TMR system could provide higher reliability than a dual modular redundancy.

The work presented in [12] proposes a system for reliability- and accuracy-aware optimization of approximate computational kernels that run on approximate hardware platforms. By defining a set of specification in the system, the proposed mechanism synthesizes the optimum hardware for running the computational kernel that achieves the target reliability and accuracy. However, this method aims at producing an approximate hardware platform.

An error resilient architecture for multicore processor has been proposed in [13] that uses a mixture of unreliable and

Authorized licensed use limited to: UNIV OF HAWAII LIBRARY. Downloaded on November 08,2025 at 00:26:35 UTC from IEEE Xplore. Restrictions apply.

reliable cores. The unreliable cores are targeted to execute the applications that are more tolerant to imprecision. This work uses fault tolerant cores with micro-architectural redundancies.

Work in [14] has defined application-specific metrics that are integrated into the application for optimizing the coverage and reliability of the approximation method.

A tool has been developed in [15] to automate the process of analyzing error effects at instruction level in order to identify approximable subset of instructions on the general purpose programs. That work is similar to the LEXACT tool presented in our paper though our tool covers the embedded world at higher levels of abstraction.

Authors in [16] have evaluated approximate probabilistic adders from the reliability point of view using a set of metrics. For this purpose, different metrics have been proposed and evaluated in this work. A number of work [17]–[20] have proposed circuit-level methods to optimized TMR system using approximation. In [17], a selective fault tolerance has been suggested that implements a smaller combinational circuit in parallel with the main one to implement an optimized TMR at circuit level. In this method, when the inputs to the circuit are not critical it is guaranteed that at least one of the smaller circuits produce the desired output so that the voting mechanism works properly. This approach relies on implementation of a modified circuit that reduces the area overhead imposed by a traditional TMR.

Authors in [18] propose a technique based on heuristics to tolerate faults in SRAM-based FPGAs by using inexact modules in conjunction with TMR, in order to reduce the area and power overhead of the design. Another work [19] proposes approximate logic circuit to implement partial TMR in FPGAs in an attempt to provide a balance between reliability and the overhead.

Authors in [20] propose an approximate TMR method based on Boolean functions for protection of circuits used in computational part of a system. However, all these work are at circuit level, and propose modifying hardware circuits to leverage the reliability of the system.

Compared to the above-mentioned methods, our proposed method's orientation is the homogenous multicore processors used in critical embedded systems. Neither modification to the microarchitecture of a core nor a modified circuit is suggested in this work. Instead, the method takes advantage of identical cores in a multicore processor to implement the redundancy, based on a software approach. Therefore, the main advantage of this method is that it does not need a new dedicated/customized hardware/circuit.

The circuit-based approaches use approximate implementation of the logic as compared to our approach that uses the same processing cores for both exact and inexact versions of a code. Therefore, one of the main advantages of this work is that it can work on commercial off the shelf (COTS) multicore processors in critical embedded applications. The application developer chooses to use the approximation or not. Furthermore, the method does not need to change the hardware already deployed in the field if the weight of one of the discussed constraints (reliability, energy, quality of result) changes later while the system is in use. In this respect, our proposed method is a new approach in design of the reliable systems where multicore processors are gaining momentum.

III. APPROXIMATE COMPUTING BACKGROUND

Applications in domains such as computer vision, media processing, machine learning, and sensor data analysis already incorporate imprecision into their design [21]. Large-scale data analytics focus on aggregate trends rather than the integrity of individual data elements. In domains such as computer vision and robotics, there are no perfect answers; results can vary in their usefulness, and the output quality is always in tension with the resources that the software needs to produce them. All these applications are approximate programs: a range of possible values can be considered "correct" outputs for a given input. From the perspective of an approximate program, today's systems are overprovisioned with accuracy. Since the program is resilient, it does not need every arithmetic operation to be precisely correct, and every bit of memory to be preserved at the same level of reliability.

A. APPROXIMATE COMPUTING METHODS

Several software and hardware-based methods have been introduced in the literature that take advantage of approximate computing techniques in favor of increasing performance and reduction in energy. Among them, only the CPU-based methods are shortlisted here since the approach taken in the present work falls under this category.

A few methods including [22] and [23] discuss the change in precision (bit-width) of input or intermediate operands to reduce storage/computing requirements. Some techniques including [24] use the 'loop perforation' approach, in which some iterations of the computational loops are skipped to reduce execution time and energy. The 'memoization' approach works by storing the results of functions for later reuse, with identical function/input. By reusing the results for similar functions/inputs, the scope of memoization can be enhanced at the cost of possible approximation. This approach is used by several approximate computing techniques such as those presented in [25] and [26]. Several approximate computing techniques for example [27], selectively skip memory references, tasks, or input portions to achieve efficiency with bounded quality of result loss. Some approximate computing techniques such as [28] utilize multiple versions of the application code considering different trade-offs between accuracy and overhead. Using inexact or faulty hardware has been the approach taken by some work such as [29] and [30] that introduce inexact adders or multipliers leading to faster critical path and smaller area in design of CPU, though being not exact in some less likely cases.

B. APPROXIMATION EVALUATION METRICS

Use of a suitable quality metric is extremely important to ensure correctness of output and balance the quality loss with reliability and efficiency gains. Based on the applications used in this

Authorized licensed use limited to: UNIV OF HAWAII LIBRARY. Downloaded on November 08,2025 at 00:26:35 UTC from IEEE Xplore. Restrictions apply.

work, two different quality metrics will be used: 1) average relative error, and 2) image difference. For the applications that produce numeric outputs (such as FFT), average relative error is used to measure the discrepancy between the original and approximate outputs. Image difference calculates the average root-mean-square of the pixel differences of the precise and approximate output images. Mean Square Error (MSE) and Peak Signal to Noise Ratio (PSNR) are two quantitative metrics mainly used to evaluate the results of such applications. In the image processing domain, a PSNR value greater than 30 dB, and in typical error-resilient applications, average relative errors less than 10 percent, are generally considered acceptable [31].

IV. SYSTEM MODELS

A hard real-time multicore system with M cores, running a set of n independent concurrent tasks $\mathrm{T} = \{\tau_1, \tau_2, \ldots, \tau_n\}$ is assumed. Tasks are scheduled according to their deadlines (e.g., by means of a Partitioned Earliest Deadline First (EDF) scheduler (P-EDF)). Each task τ_i has been marked by the application developer to inform that the task has an inexact version τ_i' with Worst-Case Execution Times (WCETs) T_{τ_i} and $T_{\tau_i'}$, respectively. Each task τ_i has a reliability requirement level $\varphi_{\tau i}$ that is annotated by the developer inside the program.

A. FAULT MODEL

Any single/multiple transient faults in logic or single/multiple upsets in system's memory may prevent the processing element, i.e., a core, from correct execution of its assigned task. However, this paper does not directly deal with such low-level phenomena. The fault model in this paper is a failure of a core from generating the expected result of a task that could exhibit itself as a processor crash, an infinite loop, or an error in results written to the memory or system's output interface.

The model considers any fault preventing a core form generating the expected results. This model takes into account the control flow errors that may cause a core to crash or go into infinite loop. However, we are protecting the system using NMR by which a specific number of failed cores could be tolerated. In other words, our fault-tolerant method uses NMR through homogenous cores in the system and the high-level fault model corresponds to this approach.

B. ENERGY MODEL

Total power consumption of a task P_τ running on a core is defined based on Eq. (1) with P_d and P_s depicting the dynamic and static portion of the total power consumption, respectively. The static component is not the focus of this paper, as it is not affected by the proposed method.

As shown in Eq. (2), the dynamic power of a core is proportional to its voltage and frequency. The term C_{eff} shows the effective capacitance that is, the product of the total capacitive load and toggling rate of the logic caused by the task,

$$P_\tau = P_d + P_s \tag{1}$$

$$P_d = C_{eff} \cdot V_{DD}^2 \cdot f \tag{2}$$

$$E_\tau = P_d \cdot T_\tau. \tag{3}$$

The energy consumption of a task is the product of its dynamic power consumption and execution time as shown by Eq. (3). Therefore, if execution time of the task is reduced then the energy consumption will be reduced linearly. Moreover, reduction in execution time of the task increases the feasibility of its re-execution in case of the failure of the core from properly accomplishing the task due to a fault, provided that the task's deadline is met.

V. PROPOSED METHOD

Traditional NMR mechanism, if implemented on a multicore processor for the reliability purpose, will consume N out of M cores in order to assure reliability of a task. In other words, this type of simple implementation needs $N - 1$ additional cores besides the main one, resulting in area and energy overheads.

In this paper, a reliability improvement method is presented that exploits the benefits of approximation in software for particular range of applications, and reduces the energy consumption and area overhead of the NMR system. The critical tasks that need higher levels of reliability, have exact and approximate versions provided by the application developer. Reliability is improved through NMR by using a combination of exact and approximate versions of the task running on K cores. Approximation of the critical tasks is performed based on the fact that, these tasks produce the result with sufficient quality even if approximation is applied.

A. LESS EXACT REDUNDANT EXECUTION

Figure 1(b) illustrates the general idea of the method for a 5MR system. Main core C_m is running the exact version of the critical task. To implement the NMR it is not necessary to have $N - 1$ cores in addition to the main one instead, the second, third and (N − 1)th redundant cores will run as much inexact version of the critical task as possible. In Figure 1(b), cores C_{m+1} and C_{m+2} each run two copies of the main task τ_{j+1}' to τ_{j+4}'. Task τ_j has a WCET of T_{τ_j}. However, approximate version of this task, i.e., τ_j' has WCET of $T_{\tau_j'}$ less than T_{τ_j}. To this end, instead of five cores, there is a need for only three cores to implement the 5MR system, with less consumed energy as compared to the exact 5MR system (Figure 1(a)). However, for a real-time system there are conditions that should be fulfilled before running multiple copies of the inexact version of the task. These are discussed in Section V.B.

Inexact copies of the tasks, i.e., τ_j' s produce approximate results as compared to the main task τ_m. Sequential execution of redundant tasks may create four different cases that have been illustrated in Figure 2. The cases have been simplified for 5MR (N = 5 in NMR) for better illustration without loss of generality:

1) EARLY FINISH OF APPROXIMATE REDUNDANTS

Figure 2(a) shows the case when the aggregate execution times of k inexact redundant copies are less than WCET

Authorized licensed use limited to: UNIV OF HAWAII LIBRARY. Downloaded on November 08,2025 at 00:26:35 UTC from IEEE Xplore. Restrictions apply.

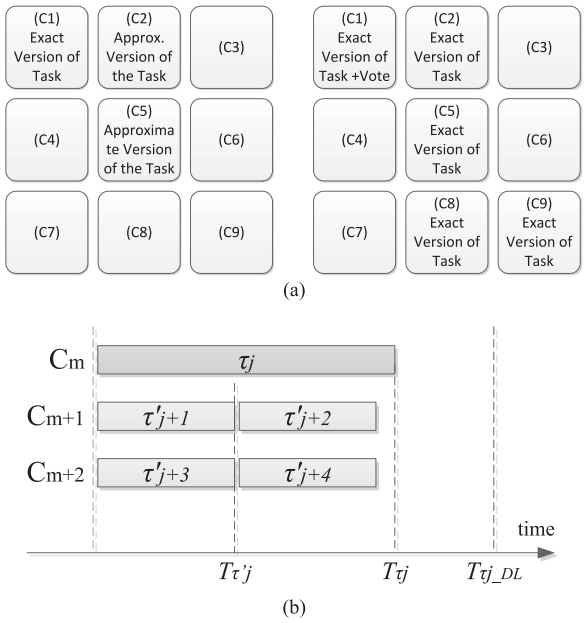


FIGURE 1. General illustration of the LEXACT method: (a, left) a 9-core system with NMR fault tolerance with configuration (N = 5) using only 3 cores (a, right) traditional NMR with involvement of 5 cores, (b) Illustration of the method from scheduling point of veiw.

of the main task, that is, $k.T_{\tau'_j} < T_{\tau_m}$. After τ_m is finished, the approximate voting happens and the result is produced. In this case, there could be enough time left on core C_{m+q}; $q = 1, 2, \ldots, k$ to implement a Duplication With Comparison (DWC) to see if the C_m has encountered any transient fault. If the result of DWC shows an error then the results of τ'_j on that specific core will not contribute to the final voting. By using the single transient fault model, the output of task τ_m could be used as the final result and no voting will be incorporated to save the energy and time.

2) EQUAL FINISH OF APPROXIMATE REDUNDANTS AND MAIN

In this case, which has been depicted in Figure 2(b), the aggregate execution time of approximate redundant tasks in a single core and execution time of the main task are rather equal, i.e., $k.T_{\tau'_j} \approx T_{\tau_m}$ and there might not be enough time to execute a comparison on the core $C_{m+q;}$ $q = 1, 2, \ldots, k$. In such a case, just an approximate voting will be performed among all tasks.

3) LATE FINISH OF APPROXIMATE REDUNDANTS

Figure 2(c) shows the case where approximate redundant tasks, overall, finish their job later than the main one, i.e., $T_{\tau_m_DL} > k.T_{\tau'_j} > T_{\tau_m}$. However, the execution time of a single approximate redundant task is still less than the main task's execution time ($T_{\tau'_j} < T_{\tau_m}$), and aggregate execution time of the redundant tasks on a single core is less than deadline. In such a case, the voting can be performed after the final task τ'_j is finished on a core C_{m+q}. In order to save more energy, it is possible to compare the results of τ_m and τ'_j and if they are within the acceptable range based on approximate

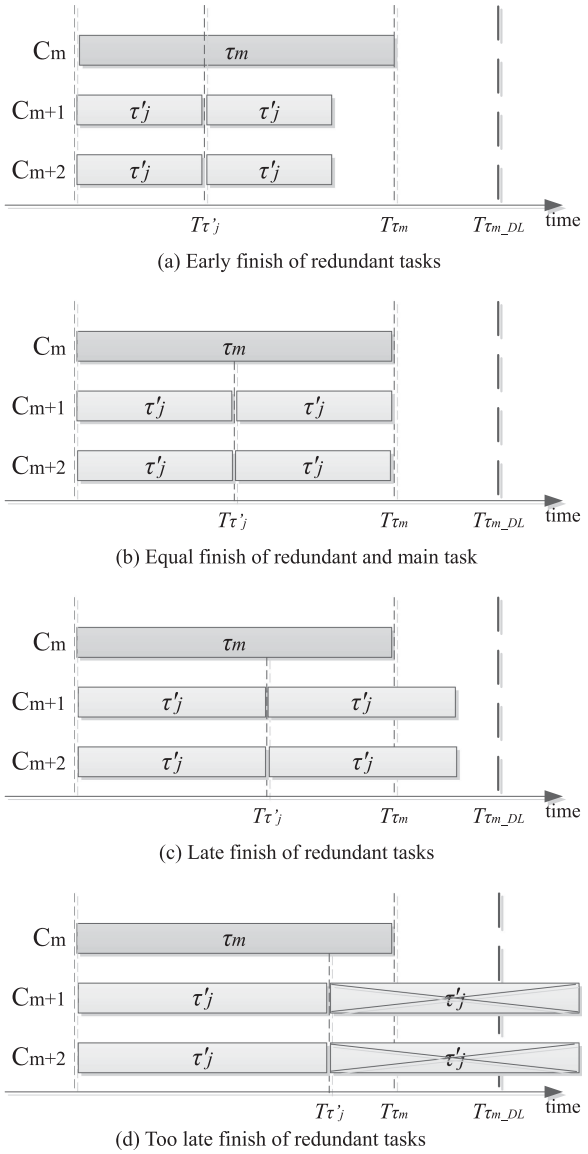


FIGURE 2. Different states of the LEXACT modular redundancy.

comparison, then the next task τ_j will be eliminated from the queue, and the result of core C_m will be used as the final result.

4) TOO LATE FINISH OF APPROXIMATE REDUNDANTS

It is necessary to finish the main and redundant tasks before the deadline. However, it might happen that the aggregate duration of redundant tasks are even larger than the deadline. In this case, redundant tasks on a core are executed sequentially, and they finish before the deadline but the last approximate redundant task will violate the deadline. Figure 2(d) shows this case for which $T_{\tau_m_DL} > (k-1) \cdot T_{\tau'_j} > T_{\tau_m}$ but $k \cdot T_{\tau'_j} > T_{\tau_m_DL} > T_{\tau_m}$. In such a case, the system is turned into reduced number of redundant tasks on each core, i.e., $(k-1)$. If the number of valid redundant tasks is reduced to $(k-1) \cdot p$ then the before-mentioned case #3 will be applicable.

Authorized licensed use limited to: UNIV OF HAWAII LIBRARY. Downloaded on November 08,2025 at 00:26:35 UTC from IEEE Xplore. Restrictions apply.

B. APPROXIMATION VERSUS RELIABILITY TRADEOFF

Apart from the benefits of approximate computing, the aim of this method is principally to provide a higher level of reliability through spatial and temporal redundancies. In other words, application of approximation should not result in an unacceptable quality or further susceptibility to faults. For this reason, a quality threshold Θ and approximation factor for a task τ', i.e., $\delta_{\tau'}$ are assumed such that the quality $\xi(\delta_{\tau'})$, that is a function of approximation factor, stays equal to or higher than this threshold, that is,

$$\xi(\delta_{\tau'}) \geq \Theta. \tag{4}$$

The approximation factor δ is the proportion of the WCET of the original version of the task τ_i to WCET of its inexact version, that is:

$$\delta_{\tau'} = {}^{T_{\tau_i}}\!\Big/_{T_{\tau'_i}}. \tag{5}$$

Furthermore, the sum of execution times of redundant tasks should not exceed the deadline of the exact task since the critical task in the system has hard real-time constraint, that is:

$$k \cdot T_{\tau'_i} < T_D. \tag{6}$$

Meanwhile, the specific level of reliability $\Phi_{\tau i}$ is required for the task that should be fulfilled through running $n-1$ copies of the original task on other processors. Therefore, reliability of the task τ_i, is a function of number of task copies n, approximation factors of the inexact version of the task $\delta_{\tau'}$ and the fault rate λ' that could be affected by approximation and is proportional to the original λ and the original worst-case execution times of the task T_τ as shown by Eq. (7).

$$R_{\tau_i} = f(N,\ \delta_{\tau'},\ \lambda',\ T_\tau) \geq \Phi_{\tau i}. \tag{7}$$

The value of approximation factor δ should be chosen so that the inequalities (4), (6), and (7) are satisfied. Table 1 shows the parameters used in the evaluation of the method.

C. METHODOLGY

In order to find a suitable approximation factor that satisfies the conditions mentioned in Section V.B., a tool called LEXACT has been developed.

LEXACT as depicted in Figure 3, primarily searches for the loops inside the application code, mainly the processing sections, then checks how reducing the loop iteration times (i.e., performing loop perforation) affects the output by checking defined evaluation metrics. Then, if reducing the iteration number of the selected loop(s) satisfies the threshold values for predefined metrics, i.e., quality of result (Θ) and deadline (T_D), then the defined reliability threshold ($\Phi_{\tau i}$) is checked through a fault injection on input data set. The injected noise on input data is considered as multiple event upsets on memory or transmission line(s) that is connected to the multicore processor.

After fault injection, the output of application is checked to see if the quality of result is above the predefined threshold; as shown in Eq. (4). If the answer is ok, then the chosen

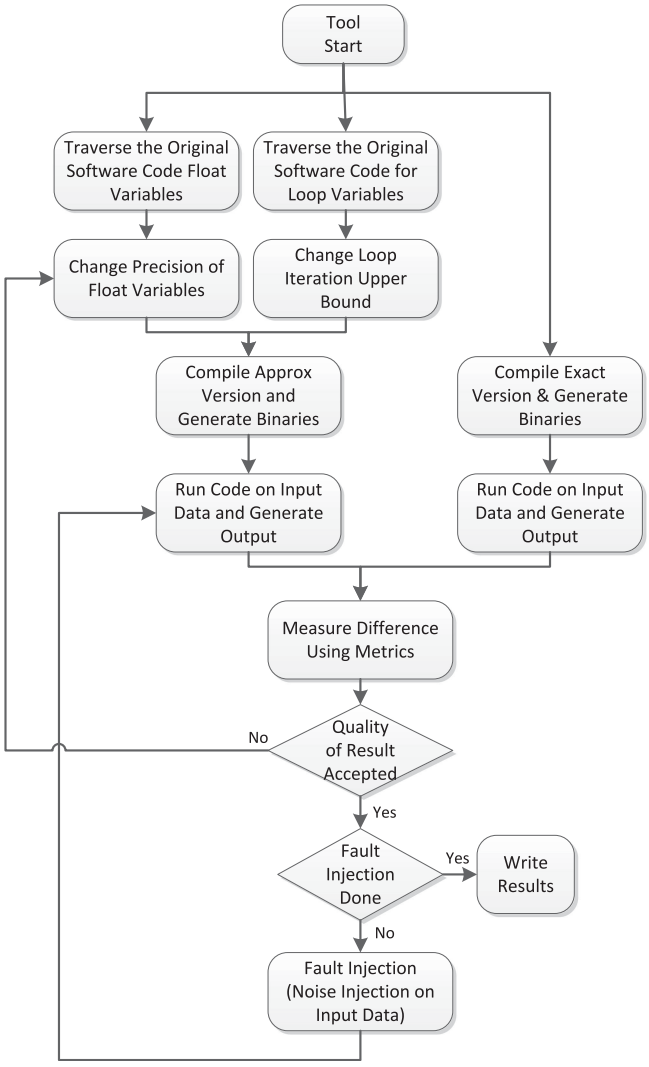


FIGURE 3. LEXACT tool operational procedure.

maximum iteration for the loop(s) will be chosen as acceptable, and the approximation factor is calculated based on Eq. (5). If the threshold of the quality of result is not satisfied then the loop perforation value is decreased gradually so that the reliability threshold is achieved.

Error injection in input data has been used to set the threshold for approximation factor. That means we have identified an approximation factor below which more fault injection onto the input data could cause the NMR reliability to drop below the defined desired value or the quality of result becomes unacceptable. In other words, in an NMR system the total reliability depends on the R(t) (Reliability) of each module that in turn depends on the fault rate. The fault rate of the modules running approximate version of the algorithm is observed using fault injection on input data. If the difference between the reliability of exact and approximate versions is below the reliability threshold then the approximation applied to the algorithm (through loop perforation or changing variable precision) is relaxed (i.e., the exactness will be increased gradually).

Authorized licensed use limited to: UNIV OF HAWAII LIBRARY. Downloaded on November 08,2025 at 00:26:35 UTC from IEEE Xplore. Restrictions apply.

Apart from loop perforation that might be useful in some iterative and recursive applications, reducing critical variable precision is another approach taken by LEXACT to check for the effect of approximation on quality of result. For example in an FFT application changing the precision of a variable has been employed instead of loop perforation. This method might be orthogonal to the loop perforation for some applications. Therefore, the main criterion for selection of the algorithms is to check for the computational kernels inside algorithm that can benefit from approximation methods.

In this method, the required reliability level is defined by the system developer who customizes the system software for the target application. Since the fault rate affects the reliability, a user-defined density of fault is injected into the input data. This fault injection could emulate the occurrence of faults for some sections of the system such as primary interfaces, system memories, and transmission path. As mentioned earlier, the NMR mechanism tolerates a specific number of core failures; hence, the fault injection on the multicore logic itself is not considered here. The fault injection on the input data is performed to set the approximation factor by which the desired quality level is also achieved. The quality level (Output Error) is determined through the maximum error on the output below which the result is unacceptable. For example, in *Susan* application (one of the benchmarks that will be discussed later in Section VI.A.) the minimum acceptable Peak Signal to Noise Ratio (PSNR), as the quality threshold, is set to 30 dB. Therefore, we inject 10 percent noise (arbitrary selected) to the input image while reducing the iteration time of the computational loop inside the algorithm kernel. By continuously calculating the PSNR, comparing it to the threshold, and reducing the loop iteration times, we achieve approximation factor of 1.4 (proportion of the execution time of the original algorithm to its approximated version) above which the quality of result would no longer be accepted.

The required quality of result, reliability level, and the rate of fault (percentage of noise injected into input data) are the inputs to the tool. The output is the suggested values of iteration times of the computational loops, which corresponds to the approximate version of the software code. The approximation factor is calculated by execution of the approximate code and comparing its execution time with the original code. The required reliability level is set by the system designer as well. The estimated fault rate (depending on such factors as surrounding environment and architecture vulnerability) is set by the developer based on which the number of N in the NMR is defined to achieve the target reliability level.

VI. EVALUATIONS AND DISCUSSION

A. BENCHMARKS

For the purpose of evaluation, a set of applications have been selected from *AxBench* [9] and *MiBench* [10] suites. *AxBench*, is a general, diverse and representative set of benchmarks for CPUs, GPUs, and hardware design. It covers a diverse set of domains such as financial analysis, machine vision, medical imaging, machine learning, data analytics, scientific computation, signal processing, image processing, robotics, and compression. Furthermore, to enable a wide range of studies, each benchmark is paired with three different input datasets. *AxBench* also provides necessary annotations to mark the approximable regions of codes and provides the application-specific quality metric to assess the output quality of each application. *MiBench* is a benchmark suite that is specifically designed for evaluation of the embedded systems. The selected applications from these benchmark suites are briefly explained in the followings:

K-means is widely used in machine learning and data mining applications. It aims at partitioning a number of n-dimensional input points into k different clusters. Each point belongs to the cluster with the nearest mean. It uses an RGB image as the input. The output is an image that is clustered in different color regions. This algorithm could be also used in safety-critical automotive application as it is discussed in [32].

Black-Scholes is a financial analysis workload. It solves partial differential equations to estimate the price for a portfolio of European options. Each option consists of different floating-point values, and the output is the estimated price of the option.

FFT is an algorithm that is used in many signal-processing applications. FFT computes the discrete Fourier transform of a sequence or its inverse. The input is a sequence of signals in time domain, and the output is the signal values in frequency domain. FFT is recently discussed, for example in [33] and [34], for the radar system used in autonomous cars.

Susan is an image recognition application that extracts the corners and edges in Magnetic Resonance Images of the brain. It is used for vision-based quality assurance, and performs adjustments for threshold, brightness, spatial control, and image smoothness. The small input data is a black-white image of a rectangle while the large input is a complex picture. *Susan* is representative of the similar algorithms that are used for feature detection in automotive domain [35].

Jmeint is a 3D gaming workload. The input is a pair of two triangles' coordinates in the 3D space, and the output is a Boolean value which indicates whether the two triangles intersect or not. This benchmark is selected to demonstrate a case for which our approximation method is not effective.

ADPCM CODEC Adaptive Differential Pulse Code Modulation (ADPCM) is a variation of the well-known standard Pulse Code Modulation (PCM) for voice. A common implementation takes 16-bit linear PCM voice samples and converts them to 4-bit samples, yielding a compression rate of 4:1. The input data are small and large speech samples.

Kalman filtering is used for many applications including filtering noisy signals, generating non-observable states, and predicting future states. This algorithm is used for autonomous cars and other critical application that require to track moving objects [36].

B. EXPERIMENTAL SETUP

For the purpose of experimental evaluation, ESESC [37], a cycle-accurate multicore simulator has been used. This

Authorized licensed use limited to: UNIV OF HAWAII LIBRARY. Downloaded on November 08,2025 at 00:26:35 UTC from IEEE Xplore. Restrictions apply.

TABLE 1. Definition of parameters used in analysis.

Parameter	Description
τ'	Inexact redundant task
Θ	Quality threshold
τ	Main version of task
δ	Approximation factor
T_D	Deadline of a task
$T_{\tau'_i}$	WCET of the inexact task i
$\Phi_{\tau i}$	Required reliability level for task i
n	Total number of cores for implementing traditional NMR
k	Total number of approximated tasks to run on a single core
$p = (n-1)/k$	Number of redundant cores in new NMR method

simulator is based on a Time-Based Sampling framework specifically developed for multicore processors that integrates power and temperature estimation tools. Another advantage of this tool is the fact that it could support setting of affinity for controlled execution of parallel applications.

In this experiment, ESESC emulates an ARM7-based homogenous multicore processor. The simulator is configured for an 8-core processor, and the operating frequency of each core is 1700 MHz.

C. ACHIEVED RESULTS

Table 2 shows the result of implementation of NMR using LEXACT tool and applying approximation on the selected applications. The selected applications have gone through a preprocessing step provided by the LEXACT tool. Loop Perforation or Precision Reduction gains between 1.35 to 2.7 times reductions in execution time while preserving the quality of result within the desirable boundaries. For example, the acceptable quality reduction for image processing algorithms (e.g., *Susan*) is chosen as PSNR above 30 dB. That means, if incorporation of approximation or inducing noise in input data decreases the quality to this level, it could be acceptable.

Loop perforation works for most of the selected applications, however for some sort of applications such as *FFT* manipulating precision of the variables significantly affects the execution time of the application. This also holds for *ADPCM* application that results in different quality of constructed digitized voice with different sampling ratios.

Another observation is that for some applications such as *Jmeint*, it is hard to find the suitable approximation factor that could satisfy all constrains including reliability, timeliness, energy and quality.

Reducing the execution time of an application will extend the opportunity for redundant execution of the task before the deadline. Most of the applications exercised in this work fall into the case #3 of "late finish of approximate redundants" as discussed in Section V-A. However, this could be due to the fact that, only the trivial algorithmic-based approximation methods such as loop perforation at the level software have been used on the selected applications. In other words, using other sophisticated hardware or software level methods for approximate computing may result in further overhead reduction and realization of cases #1 or #2.

TABLE 2. Effects of approximation on energy consumption and quality of results.

Bench Name	Computation Type	Approx. Type	Cycle Count	Execution Time (ms)	Approx. Factor δ	Energy Saving for NMR (%)	Input	Output Error	Error Evaluation Metric	No. of Core for NMR (N = 3 / TMR)	Tool Computation Time (sec)
Susan	Exact	–	118382700	69	–	–	Healthy / Noisy	99 dB / 50.74 dB	PSNR	3	11.5
	Approx.	Loop Perforation	83773000	49.3	1.40	19%	Healthy / Noisy	41.44 dB / 38.44 dB		2	
FFT	Exact	–	146632115	86.254	–	–	Healthy / Noisy	0	Relative Error	3	6
	Approx.	Variable Precision Reduction	105952826	62.325	1.38	18%	Healthy / Noisy	1.2%		2	
BlackScholes	Exact	–	317811653	186.948	–	–	Healthy / Noisy	0	Relative Error	3	38
	Approx.	Loop Perforation	161009877	94.712	1.97	32%	Healthy / Noisy	3.5%		2	
k-means	Exact	–	609583254	358.578	–	–	Healthy / Noisy	98 dB / 44.8 dB	PSNR	3	21.2
	Approx.	Loop Perforation	225239057	132.494	2.71	36%	Healthy / Noisy	38.3 dB / 33.5 dB		2	
Jmeint	Exact	–	172317611	101.36	–	–	Healthy / Noisy	98 dB / 28.5 dB	PSNR	3	45
	Approx.	Loop Perforation	171042442	100.61	1.01	1%	Healthy / Noisy	26.3 dB / 22.1 dB		2	
ADPCM-CODEC	Exact	–	285601524	168.12	–	–	Healthy / Noisy	98 dB / 52.8 dB	SNR	3	31.4
	Approx.	Sample Data Reduction	210810147	124.32	1.35	28%	Healthy / Noisy	38.3 dB / 33.5 dB		2	
Kalman Filter (Embedded Extended-Kalman)	Exact	–	537892987	316.41	–	–	Healthy / Noisy	99 dB / 50.74 dB	PSNR	2	46.2
	Approx.	Loop Perforation	378797878	222.82	1.42	33%	Healthy / Noisy	41.44 dB / 38.44 dB		3	

Authorized licensed use limited to: UNIV OF HAWAII LIBRARY. Downloaded on November 08,2025 at 00:26:35 UTC from IEEE Xplore. Restrictions apply.

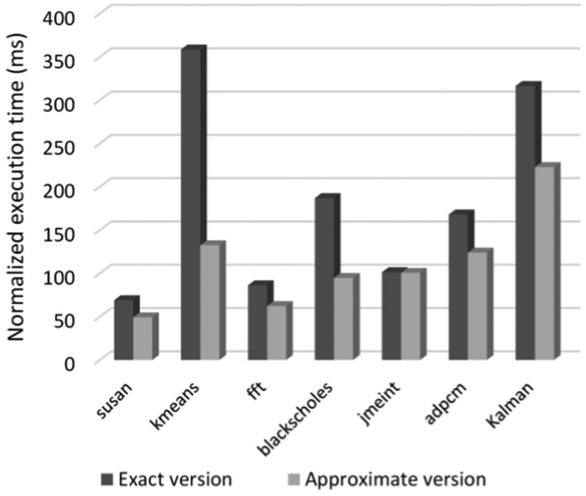


FIGURE 4. **Execution time of applications: (dark gray) exact version, (light gray) approximate version.**

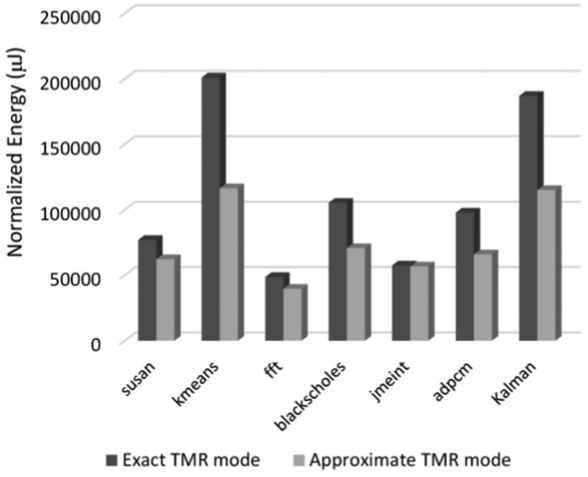


FIGURE 5. **Energy consumption of TMR configuration of multicore processor: (dark gray) Running tripple exact copies of the application on different processors, (light gray) after use of approximation using two cores for realization of TMR.**

The reduction in execution time, causes the NMR (in these experiments: N = 3) needing two cores instead of three for the reaching the same reliability level. This results in 33 percent reduction in the area of NMR system, with the same reliability level. The reduction in execution time of each application has been shown in Figure 4. The approximation factor varies between 1.35 to 2.7 that provides enough room for implementation of NMR using less number of cores.

The energy savings achieved by this method is between 18 to 35 percent as shown in Figure 5 through applying the method on the selected benchmarks for the TMR configuration.

The proposed method works at thread-level, and the selected benchmarks are considered to be a number of independent tasks that are to be executed on a multicore platform. Therefore, the granularity is so that it is not required to work on large application source codes even though it is feasible. In other words, it is up to the system developer to define the granularity of the code which is to be approximated. Basically, the selected codes have a computational part (kernel) that can be approximated which is what we focus on. In fact, the LEXACT tool does not work on-line and it is intended to generate the approximate versions offline. Therefore, its runtime does not affect the embedded system itself. The main duty of this tool is to generate the approximate version of a code (a computational kernel) based on the input parameters (approximation, quality, and reliability factors), and try to converge to an optimum point, if one exists. The computation time of the tool for generating the optimum result for the given applications is shown in the last column of Table 2.

D. EFFECTS OF APPROXIMATION ON RELIABILITY

Approximate computing causes the quality of result to be reduced and the application to be more susceptible to the faults as discussed in [38] and [15]. The algorithms/applications with capability of approximation are good at tolerating faults. In other words, an approximable code can tolerate faults due to its inherent fault tolerance. However, in this work we intentionally squeeze the approximable part of the code in favor of reducing execution time and its energy consumption. Therefore, computation of the algorithm in an approximate manner with lower quality of result leaves less room for further fault tolerance. Consider the case when a floating-point variable precision is reduced in favor of reducing the computational time. If this variable is affected by some faults so that the precision is further reduced then the algorithm's output will be undesirable from the quality of result perspective. In this regard, the intentional use of approximation makes the application/algorithm more susceptible to faults.

It is necessary to evaluate the effect of inducing faults on the exact and approximate versions of the applications and compare the results. LEXACT tool provides a means for applying noise on input data. This translates to injecting faults on the memory holding the data or communication line transferring the data to the processing units.

The tool carries out the fault injection via inducing a random noise on the input data. Then it evaluates the exact and approximate versions of the target application for both healthy and noisy input data with respect to the quality of result. The approximation factor (δ) is chosen so that the quality of result for both cases are close, according to the threshold defined by the system designer (as an input parameter to the tool). This has been shown in Table 2 as the 'Output Error' column showing the error result for healthy and noisy input data.

Figure 6 shows a visual comparison made for *Susan* application processing a large input image. Figure 6(a) shows the original image while Figure 6(b) is the same image vitiated with a random noise. Executing the application in corner detection mode on Figure 6(a) results in detection of some 490 corners in the picture. Running the exact version of the application on Figure 6(b) results in detection of almost 490 corners out of which 65 cases are false. Applying approximate version of the application on the original image

Authorized licensed use limited to: UNIV OF HAWAII LIBRARY. Downloaded on November 08,2025 at 00:26:35 UTC from IEEE Xplore. Restrictions apply.

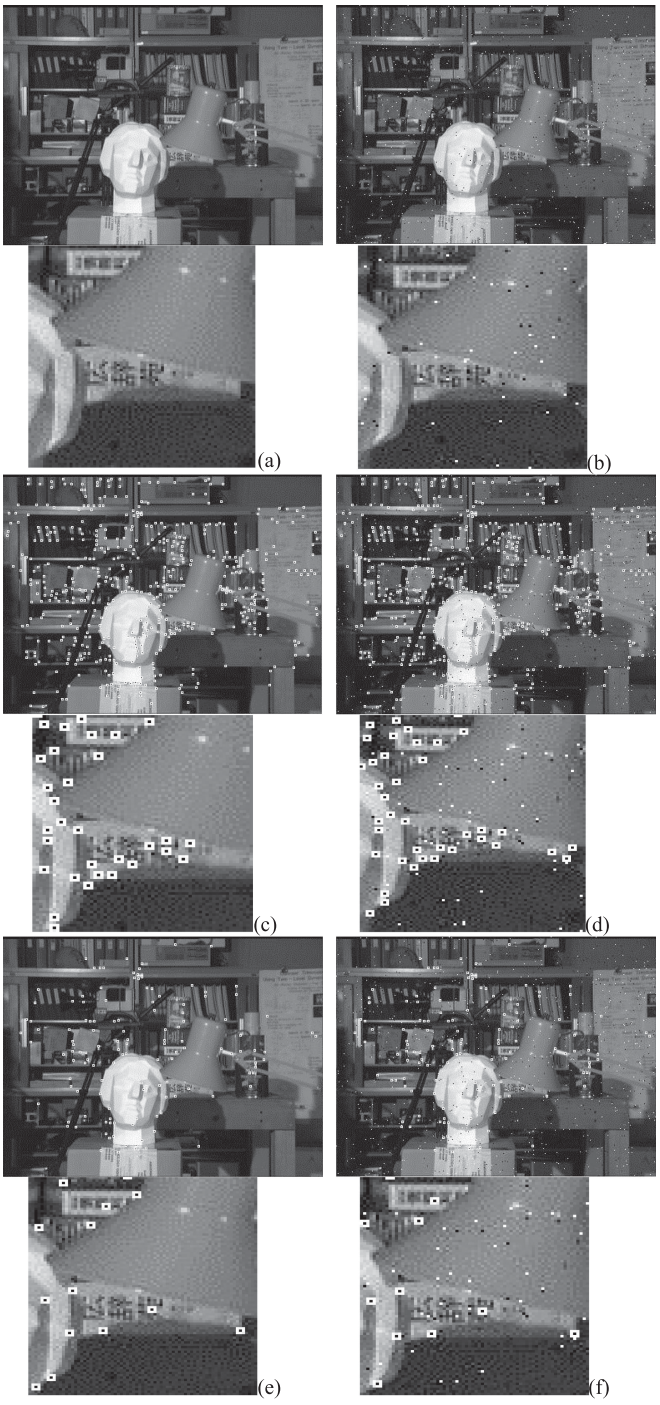


FIGURE 6. Quality of result degradation comparison: (a) Original input image, (b) Noisy input image, (c) Exact computation of original image, (d) Exact compuation of noisy image, (e) Approximate computed original image with $\delta = 1.40$, (f) Approximate computed noisy image with $\delta = 1.40$ and Err $= 38.44$ dB.

while providing 40 percent speedup, it detects some 290 correct corners that is acceptable according to the predefined quality metric as shown in Figure 6(c). Figure 6(d) shows the result of approximate computation of the algorithm on a noisy image (Figure 6(b)), which results in 275 correctly detected corners.

E. LIMITATION OF THE METHOD

The main criteria of selection of the algorithms is to check for the computational kernels within the algorithm that can benefit from approximation methods. For example, algorithms with iterative loops or recursive algorithms are candidates for this method. Moreover, if precision of some variables within an application is so large that can be reduced in favour of speeding up the execution time, then this method is applicable. Therefore, the method cannot be applied to all applications used in a critical system unless an approximate/inexact computation is feasible.

VII. CONCLUSION AND FUTURE WORK

Approximate computing is finding its key role in reducing the energy consumption in a variety of applications especially in embedded systems. A new trend is to control the energy and area overhead induced by the reliability methods using this methodology. In this paper, a method is introduced for reducing energy and area overheads associated with NMR as a means for reliability improvement in multicore processors. This method helps preventing overcalculating the number of cores in an embedded multicore processor when designing for reliability in critical embedded systems. A tool called LEXACT has been developed to process the selected applications with respect to an approximation factor that can satisfy the quality of result, real-time constraint, and the reliability level of system. Finally, a set of approximable applications has been chosen to evaluate the method.

In this work, the main core in a multicore processor executes the exact version of code. This approach has been selected in order to provide a base for future research on the architectural methods that use heterogeneous multicores comprising of cores with precise and approximate logics. This could further reduce the energy consumption and area while reducing the complexities associated with algorithmic methods of approximate computing.

REFERENCES

[1] C. El Salloum, M. Elshuber, O. Hoftberger, H. Isakovic, and A. Wasicek, "The ACROSS MPSoC – A new generation of multi-core processors designed for safety-critical embedded systems," in *Proc. 15th Euromicro Conf. Dig. Syst. Design*, 2012, vol. 37, no. 8, pp. 105–113.
[2] S. Kehr, E. Quiñones, B. Böddeker, and G. Schäfer, "Parallel execution of AUTOSAR legacy applications on multicore ECUs with timed implicit communication," in *Proc. 52nd Annu. Des. Autom. Conf.*, 2015, pp. 1–6.
[3] N. Navet, A. Monot, B. Bavoux, and F. Simonot-Lion, "Multi-source and multicore automotive ECUs - OS protection mechanisms and scheduling," in *Proc. IEEE Int. Symp. Ind. Electron.*, 2010, pp. 3734–3741.
[4] M. Salehi, *et al.*, "DRVS: Power-efficient reliability management through dynamic redundancy and voltage scaling under variations," in *Proc. IEEE/ACM Int. Symp. Low Power Electron. Des.*, 2015, pp. 225–230.
[5] M. Salehi, A. Ejlali, and B. Al-Hashimi, "Two-phase low-energy N-modular redundancy for hard real-time multi-core systems," *IEEE Trans. Parallel Distrib. Syst.*, vol. 27, no. 5, pp. 1497–1510, May 2016.
[6] M. A. Haque, H. Aydin, and D. Zhu, "Energy-aware task replication to manage reliability for periodic real-time applications on multicore platforms," in *Proc. Int. Green Comput. Conf.*, 2013, pp. 1–11.
[7] F. Baharvand and S. G. Miremadi, "ARMOR: Adaptive reliability management by on-the-fly redundancy in multicore embedded processors," in *Proc. 21st IEEE Pacific Rim Int. Symp. Dependable Comput.*, 2015, pp. 215–224.

Authorized licensed use limited to: UNIV OF HAWAII LIBRARY. Downloaded on November 08,2025 at 00:26:35 UTC from IEEE Xplore. Restrictions apply.

[8] J. Han and M. Orshansky, "Approximate computing: An emerging paradigm for energy-efficient design," in *Proc. 18th IEEE Eur. Test Symp.*, 2013, pp. 1–6.

[9] A. Yazdanbakhsh, D. Mahajan, H. Esmaeilzadeh, and P. Lotfi-Kamran, "AxBench: A Multiplatform Benchmark Suite for Approximate Computing," in *IEEE Design & Test*, vol. 34, no. 2, pp. 60–68, Apr. 2017.

[10] M. R. Guthaus, J. S. Ringenberg, D. Ernst, T. M. Austin, T. Mudge, and R. B. Brown, "MiBench: A free, commercially representative embedded benchmark suite," in *Proc. 4th Annu. IEEE Int. Workshop Workload Characterization*, 2001, pp. 3–14.

[11] K. Chen and F. Lombardi, "An approximate voting scheme for reliable computing," in *Proc. Des. Autom. Test Europe Conf. Exhib. Workshop*, 2015, pp. 293–296.

[12] S. Misailovic, M. Carbin, S. Achour, Z. Qi, and M. C. Rinard, "Chisel: Reliability- and accuracy-aware optimization of approximate computational kernels," in *Proc. Int. Conf. Object Oriented Program. Syst. Languages Appl.*, 2014, pp. 309–328.

[13] L. Leem, Hyungmin Cho, J. Bau, Q. A. Jacobson, and S. Mitra, "ERSA: Error Resilient System Architecture for probabilistic applications," in *Proc. Design, Automation & Test in Europe Conference & Exhibition (DATE 2010)*, 2010, pp. 1560–1565.

[14] B. Grigorian and G. Reinman, "Dynamically adaptive and reliable approximate computing using light-weight error analysis," in *Proc. NASA/ESA Conf. Adaptive Hardw. Syst.*, 2014, pp. 248–255.

[15] R. Venkatagiri, A. Mahmoud, S. K. S. Hari, and S. V. Adve, "Approxilyzer: Towards a systematic framework for instruction-level approximate computing and its application to hardware resiliency," in *Proc. 49th Annu. IEEE/ACM Int. Symp. Microarchitecture*, 2016, pp. 1–14.

[16] J. Liang, J. Han, and F. Lombardi, "New metrics for the reliability of approximate and probabilistic adders," *IEEE Trans. Comput.*, vol. 62, no. 9, pp. 1760–1771, Sep. 2013.

[17] M. Augustin, M. Gossel, and R. Kraemer, "Implementation of selective fault tolerance with conventional synthesis tools," in *Proc. 14th IEEE Int. Symp. Des. Diagnostics Electron. Circuits Syst.*, 2011, pp. 213–218.

[18] S. Venkataraman, R. Santos, and A. Kumar, "A flexible inexact TMR technique for SRAM-based FPGAs," in *Proc. Des. Autom. Test Europe Conf. Exhib.*, 2016, pp. 810–813.

[19] A. J. Sanchez-Clemente, L. Entrena, and M. Garcia-Valderas, "Partial TMR in FPGAs using approximate logic circuits," *IEEE Trans. Nuclear Sci.*, vol. 63, no. 4, pp. 2233–2240, Aug. 2016.

[20] I. A. C. Gomes, M. G. A. Martins, A. I. Reis, and F. L. Kastensmidt, "Exploring the use of approximate TMR to mask transient faults in logic with low area overhead," *Microelectronics Rel.*, vol. 55, no. 9/10, pp. 2072–2076, 2015.

[21] S. Mittal, "A survey of techniques for approximate computing," *J. Chemical Inf. Model.*, vol. 53, no. 9, pp. 1689–1699, 2013.

[22] T. Yeh, P. Faloutsos, M. Ercegovac, S. Patel, and G. Reinman, "The art of deception: Adaptive precision reduction for area efficient physics acceleration," in *Proc. 40th Annu.IEEE/ACM Int. Symp. Microarchitecture*, 2007, pp. 394–406.

[23] Q. Zhang, T. Wang, Y. Tian, F. Yuan, and Q. Xu, "ApproxANN: An approximate computing framework for artificial neural network," in *Proc. Design, Autom. Test Europe Conf. Exhib.*, 2015, pp. 701–706.

[24] S. Sidiroglou, S. Misailovic, H. Hoffmann, and M. Rinard, "Managing performance versus accuracy trade-offs with loop perforation," in *Proc. 19th ACM SIGSOFT Symp. 13th Eur. Conf. Found. Softw. Eng.*, 2011, pp. 124–134.

[25] A. Rahimi, A. Marongiu, R. K. Gupta, and L. Benini, "A variability-aware OpenMP environment for efficient execution of accuracy-configurable computation on shared-FPU processor clusters," in *Proc. Int. Conf. Hardw./Softw. Codesign Syst. Synthesis*, 2013, Art. no. 35.

[26] G. Keramidas, C. Kokkala, and I. Stamoulis, "Clumsy value cache: An approximate memoization technique for mobile GPU fragment shaders," in *Proc. 1st Workshop Approximate Comput.*, 2015, Art. no. 6.

[27] M. Samadi, D. A. Jamshidi, J. Lee, and S. Mahlke, "Paraprox: Pattern-based approximation for data parallel applications," in *Proc. Int. Conf. Archit. Support Program. Languages Operating Syst.*, 2014, pp. 35–50.

[28] M. Samadi, J. Lee, D. A. Jamshidi, A. Hormati, and S. Mahlke, "SAGE: Self-tuning approximation for graphics engines," in *Proc. 46th Annu. IEEE/ACM Int. Symp. Microarchitecture*, 2013, pp. 13–24.

[29] A. B. Kahng and S. Kang, "Accuracy-configurable adder for approximate arithmetic designs," in *Proc. 49th Annu. Des. Autom. Conf.*, 2012, Art. no. 820.

[30] P. Kulkarni, P. Gupta, and M. Ercegovac, "Trading accuracy for power with an underdesigned multiplier architecture," in *Proc. IEEE Int. Conf. VLSI Des.*, 2011, pp. 346–351.

[31] A. Rahimi, A. Ghofrani, K. Cheng, L. Benini, and R. K. Gupta, "Approximate associative memristive memory for energy-efficient GPUs," in *Proc. Des. Autom. Test Europe Workshop*, 2015, pp. 1497–1502.

[32] R. Kluger, B. L. Smith, H. Park, and D. J. Dailey, "Identification of safety-critical events using kinematic vehicle data and the discrete fourier transform," *Accident Anal. Prevention*, vol. 96, pp. 162–168, Nov. 2016.

[33] D. T. Nugraha, A. Roger, and R. Ygnace, "Integrated FFT accelerator and inline bin-rejection for automotive FMCW radar signal processing," in *Proc. Eur. Microwave Conf.*, 2015, pp. 1586–1589.

[34] F. Baselice, G. Ferraioli, S. Lukin, G. Matuozzo, V. Pascazio, and G. Schirinzi, "A new methodology for 3D target detection in automotive radar applications," *Sensors*, vol. 16, no. 5, 2016, Art. no. 614.

[35] Y. Uk Yim and S.-Y. Oh, "Three-feature based automatic lane detection algorithm (TFALDA) for autonomous driving," *IEEE Trans. Intell. Transp. Syst.*, vol. 4, no. 4, pp. 219–225, Dec. 2003.

[36] A. Macaveiu and A. Campeanu, "Automotive radar target tracking by Kalman filtering," in *Proc. 11th Int. Conf. Telecommun. Modern Satellite Cable Broadcasting Serv.*, 2013, pp. 553–556.

[37] E. K. Ardestani and J. Renau, "ESESC: A fast multicore simulator using time-based sampling," in *Proc. IEEE 19th Int. Symp. High Performance Comput. Archit.*, 2013, pp. 448–459.

[38] X. Li and D. Yeung, "Application-level correctness and its impact on fault tolerance," in *Proc. 13th IEEE Int. Symp. High Performance Comput. Archit.*, 2007, pp. 181–192.



FARSHAD BAHARVAND received the BSc and MSc degrees both in computer engineering from University of Tehran, Iran in 1993 and 1996, respectively. He has been with the microelectronic industry since 1997. He has the experience of developing application-specific microprocessors and multicore system-on-chips for a variety of embedded applications including new generation of communication systems. He is also co-founder of SiNA Microelectronics Inc.; a fabless semiconductor company established in 2004, Iran. He is currently working toward the PhD degree in computer engineering at Sharif University of Technology. His research interests include fault-tolerant processor architectures, multicore processors for cyber-physical systems, approximate computing, and low-power digital integrated circuits.



SEYED GHASSEM MIREMADI received the MSc degree in applied physics and electrical engineering from Linköping Institute of Technology and the PhD degree in computer engineering from Chalmers University of Technology, Sweden, in 1984 and 1995, respectively. He is a professor of computer engineering with Sharif University of Technology. As fault-tolerant computing is his specialty, he initiated the "Dependable Systems Laboratory" at Sharif University in 1996 and has chaired the Laboratory since then. The research laboratory has participated in several research projects that have led to several scientific articles and conference papers. He and his group have done research in physical, simulation-based and software-implemented fault injection, dependability evaluation using HDL models, fault-tolerant embedded systems, fault-tolerant multicore processors, fault-tolerant NoCs, fault-tolerant real-time systems, and fault-tolerant storage systems. He was the education director (1997-1998), the head (1998-2002), the research director (2002-2006), and the director of the hardware froup (2009-2010) of the Computer Engineering Department, Sharif University. During 2003 to 2010, he was the director of the Information Technology Program, Sharif International Campus in Kish Island. From 2010 to 2012, he was the vice-president of Academic Affairs (VPAA) of Sharif University. Since 2014, he is VPAA of University. He served as the general co-chair of the 13th Int'l CSI Computer Conference (CSICC 2008) and general co-chair of the 2015 Int'l CSI Symposium on Real-Time and Embedded Systems and Technologies (RTEST 2015). He is currently the editor of the *Scientia Transactions on Computer Science and Engineering*. He is a senior member of the IEEE Computer Society and IEEE Reliability Society.

Authorized licensed use limited to: UNIV OF HAWAII LIBRARY. Downloaded on November 08,2025 at 00:26:35 UTC from IEEE Xplore. Restrictions apply.